AUTONOMOUS UNIVERSITY OF THE STATE OF MEXICO

FACULTY OF ENGINEERING

DEGREE IN ARTIFICIAL INTELLIGENCE ENGINEERING

SOFTWARE ENGINEERING II

INTELLIGENT RECOMMENDATION PLATFORM

DEVELOPER MANUAL

Version 2.0
Windows – MacOS - Linux

PRESENTED BY:

BETZY PERLA RAMIREZ SOLIS

FABIOLA LAUREANO MARTINEZ

MICHEL GONZALEZ BECERRIL

ANGEL GABRIEL SERRANO VAZQUEZ

OCTOBER 26th, 2026.

Development setup, architecture, data preparation, API use, testing, and maintenance.

Source revision 00cabc2

Contents

1. SCOPE AND REPOSITORY 4

2. ARCHITECTURE AND REQUEST FLOW 5

Windows 6

3. WINDOWS DEVELOPMENT SETUP 7

4. WINDOWS RUN AND INSPECT 8

MacOS 9

5. MacOS DEVELOPMENT SETUP 10

6 MacOS RUN AND INSPECT 11

Linux 12

7. LINUX DEVELOPMENT SETUP 13

8. LINUX EXECUTION EVIDENCE 14

9. DATA PREPARATION AND PERSISTENCE 15

10. RECOMMENDATION METHODS 16

11. API CONTRACTS AND EXAMPLES 17

12. ERROR HANDLING AND EVENTS 18

13. CONFIGURATION AND SERVICES 19

14. EXTERNAL SERVICES AND CACHE BEHAVIOR 20

15. ADD OR CHANGE A FEATURE 21

16. TESTS AND MEASURED PERFORMANCE 22

17. DEBUGGING AND DELIVERY 23

18. REFERENCES 24

1. SCOPE AND REPOSITORY

Use this guide to develop and maintain the local movie recommendation platform. It assumes basic Python and terminal knowledge. The commands target source revision 00cabc2 of version 0.2.0. Repository behavior and limitations in this guide are based on that revision (Michou001, 2026).

The platform includes a FastAPI service, a Spanish web interface, prepared MovieLens data, four recommendation methods, and background interaction recording. Python 3.11 or newer is required. The dependency file contains minimum versions, not a complete locked environment. Use git clone to create a local working copy (Git, n.d.).

LOCATION	PURPOSE
src/presentation	Input and output schemas; static web page.
src/application	API routes and workflow coordination.
src/ai	Strategy contracts, models, factory, and decorators.
src/data	Feature access and offline data pipeline.
src/persistence	Profiles, catalog, history, and embeddings.
src/messaging	Event queue and history worker.
src/config	Environment-based settings.
tests and benchmarks	Validation and response-time measurements.

NOTE: Use a working branch and keep generated data separate from source changes. Inspect git status before committing.

2. ARCHITECTURE AND REQUEST FLOW

application/api.py is the composition root: it creates the data stores, strategies, decorators, controller, and worker. Other layers depend on contracts. Tests check the allowed import directions and the documented composition-root exceptions.

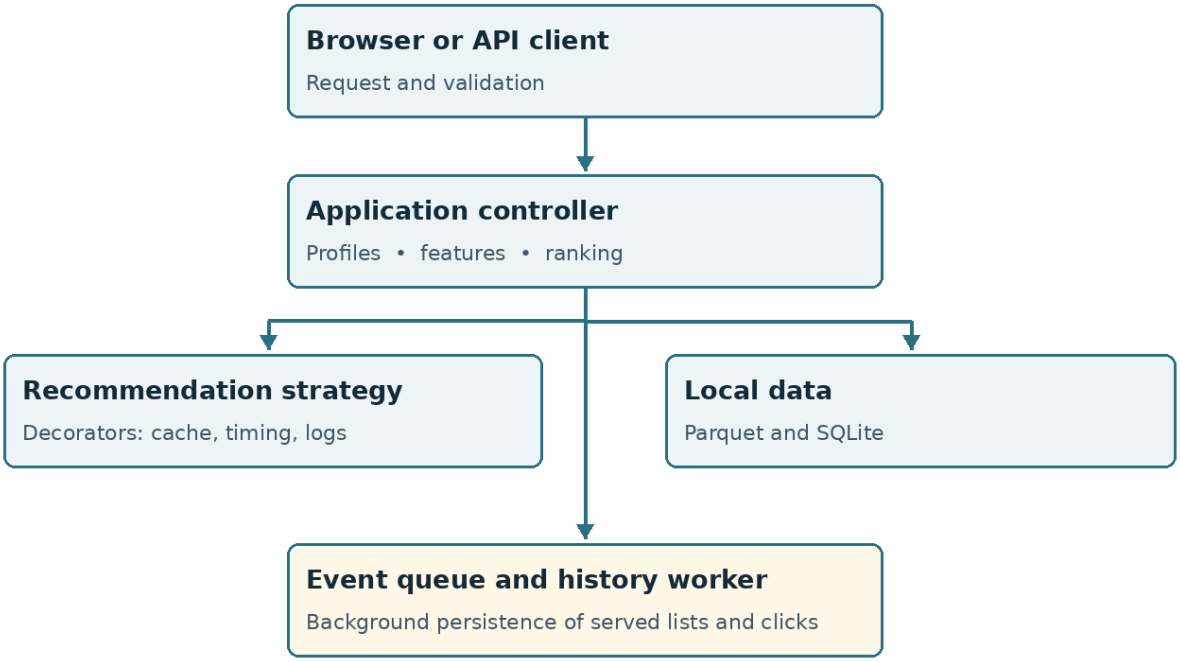


Figure 1. Main runtime components and background history storage.

For a recommendation, the controller checks the user, loads prepared features, ranks movies, adds catalog information, and emits a recommendation_served event. It returns a list plus the actual strategy, versions, latency, and degradation fields.

The browser sends interest clicks separately to POST /interactions. A memory queue is consumed by a worker thread inside the API process. A RabbitMQ queue requires a separate worker process. These events are recorded; the request path does not train or update feature vectors.

Windows

3. WINDOWS DEVELOPMENT SETUP

Use PowerShell. Install Python 3.11 or newer and Git, then reopen the terminal. The examples use the Python launcher. If py is unavailable, use a verified Python executable instead.

```
py -3 --version
git --version
git clone https://github.com/Michou001/Intelligent-Recommendation-Plataform.git
cd Intelligent-Recommendation-Plataform
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -e ".[dev,als]"
```

The editable install makes source changes available without reinstalling the package. The dev extra installs the test and code-checking tools. The als extra supplies implicit for offline ALS training. Serving existing ALS factors uses NumPy and does not itself require implicit.

3.1 Optional components

```
python -m pip install -e ".[deep]"
python -m pip install -e ".[content,cache,broker,nosql]"
```

Install deep for NCF training and inference. Install content for sentence-transformer embeddings; cache, broker, and nosql add clients for external services. Installing clients does not start their servers.

NOTE: If PowerShell blocks activation, use Command Prompt in this folder and run .venv\Scripts\activate.bat. You can also call .\.venv\Scripts\python.exe directly. No global policy change is required (Python Software Foundation, n.d.).

4. WINDOWS RUN AND INSPECT

4.1 Start the API

```
python -m uvicorn application.api:app --reload --port 8000
```

Leave the terminal open. Browse to http://127.0.0.1:8000/ for the web page and http://127.0.0.1:8000/docs for API exploration. Use Ctrl+C to stop the process. --reload is for development (FastAPI, n.d.).

4.2 Call the service from a second PowerShell window

```
Invoke-RestMethod http://127.0.0.1:8000/health
Invoke-RestMethod "http://127.0.0.1:8000/recommendations/1?k=3"
$body = @{user_id=1; event_type="click"; item_id=293}
$json = $body | ConvertTo-Json
Invoke-RestMethod -Uri http://127.0.0.1:8000/interactions `
-Method Post -ContentType "application/json" -Body $json
```

The backtick continues the PowerShell command onto the next line. The interaction call records a test event in your configured history store.

4.3 Change the method for this terminal session

```
$env:RECO_ACTIVE_STRATEGY="content"
python -m uvicorn application.api:app --port 8000
Remove-Item Env:RECO_ACTIVE_STRATEGY
```

Stop the server before changing its configuration. Run Remove-Item after stopping it to restore the default for the next start. If port 8000 is occupied, use --port 8001 and change the request URLs.

MacOS

5. MacOS DEVELOPMENT SETUP

Use Terminal with zsh or bash. Install a Python 3.11 or newer interpreter for your Mac and install Git. Check the commands before creating the environment. Create a new environment on each machine; do not copy a Windows .venv to macOS (Python Software Foundation, n.d.).

```
python3 --version
git --version
git clone https://github.com/Michou001/Intelligent-Recommendation-Plataform.git
cd Intelligent-Recommendation-Plataform
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e ".[dev,als]"
```

5.1 Optional components

```
python -m pip install -e ".[deep]"
python -m pip install -e ".[content,cache,broker,nosql]"
```

On Apple Silicon, use an interpreter and package wheels that match the machine architecture. If pip reports that a compiler or compatible wheel is missing, review the failing package and Python version before changing the project.

NOTE: If implicit cannot be installed, use python -m pip install -e ".[dev]" for the base development environment. Existing ALS factors can still be served. A new pipeline run without implicit uses the implemented SVD fallback, so its output is not an ALS training run.

The default content pipeline uses TF-IDF and SVD. It does not download a transformer model. Select sentence-transformers only when you intend to test that optional path.

6 MacOS RUN AND INSPECT

6.1 Start and stop

```
python -m uvicorn application.api:app --reload --port 8000
```

Open http://127.0.0.1:8000/ in a browser. Keep Terminal open and press Control+C to stop the API. Run the following calls in a second terminal.

6.2 Exercise the public endpoints

```
curl -sS http://127.0.0.1:8000/health
curl -sS "http://127.0.0.1:8000/recommendations/1?k=3"
curl -sS -X POST http://127.0.0.1:8000/interactions \
  -H "Content-Type: application/json" \
  -d '{"user_id":1,"event_type":"click","item_id":293}'
```

The backslash continues a shell command. Keep the JSON in single quotes so the shell preserves its double quotes.

6.3 Set environment variables

```
export RECO_ACTIVE_STRATEGY=content
python -m uvicorn application.api:app --port 8000
unset RECO_ACTIVE_STRATEGY
```

Use unset after stopping the server to return to the default for future starts. For a one-command change, use RECO_ACTIVE_STRATEGY=content before the Python command. The same shell syntax is used in the Linux section.

NOTE: Windows and MacOS commands here were checked against the source and shell syntax. The executed outputs shown later were captured on Linux; they are not evidence of an OS-specific macOS test.

Linux

7. LINUX DEVELOPMENT SETUP

Use bash or a compatible shell. On Debian or Ubuntu, install the system prerequisites with the commands below if they are missing. Other distributions use different package managers. Check that the installed interpreter is Python 3.11 or newer.

```
sudo apt update
sudo apt install python3 python3-venv python3-pip git
python3 --version
git --version
git clone https://github.com/Michou001/Intelligent-Recommendation-Plataform.git
cd Intelligent-Recommendation-Plataform
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e ".[dev,als]"
```

Use pip inside the virtual environment. If the system reports an externally managed Python installation, create or activate .venv rather than forcing packages into the system interpreter.

7.1 Start and make a request

```
python -m uvicorn application.api:app --reload --port 8000
```

In a second terminal, run:

```
curl -sS http://127.0.0.1:8000/health
curl -sS "http://127.0.0.1:8000/recommendations/1?k=3"
```

Stop with Ctrl+C. To select another strategy, use export RECO_ACTIVE_STRATEGY=content before startup. Remove the override with unset RECO_ACTIVE_STRATEGY after stopping the process.

8. LINUX EXECUTION EVIDENCE

The API and checks were executed in an isolated copy of revision 00cabc2 on Linux with Python 3.12.14. The API test used port 8765 to avoid conflicts. The images below reproduce captured console output; line wrapping and selected fields improve readability.

Linux terminal output

```
$ curl -sS http://127.0.0.1:8765/health
{
  "status": "ok",
  "active_strategy": "als",
  "feature_store_available": true,
  "cache_backend": "in-memory",
  "queue_backend": "in-memory",
  "catalog_items": 7343,
  "user_profiles": 609
}
```

Figure 2. Actual Linux API output with selected health fields.

The response reported 609 stored profiles and 7,343 catalog items. These are the prepared artifacts in this revision, not the size of the original raw dataset.

A recommendation request for user 1 returned three movies with strategy als and degraded=false. Invalid user ID 0 returned HTTP 400 with validation_error. A test click returned HTTP 202 with accepted=true and queued=true.

9. DATA PREPARATION AND PERSISTENCE

Use the included artifacts for a first run. Rebuild only when changing source data, features, or training settings. The pipeline reads MovieLens CSV data and runs ingestion, cleaning, transformation, and materialization (GroupLens Research, n.d.; Michou001, 2026).

```
python -m data.pipeline
python -m persistence --seed
```

Add --download to the pipeline command only when the selected dataset is missing and you intend to download it. You may run stages separately as python -m data.pipeline.ingest, .clean, .transform, and .materialize using the full module name each time.

OUTPUT	CONTENTS
artifacts/interim	Raw, cleaned, and transformed Parquet data plus reports.
artifacts/feature_store/v1	User and item features, popularity tables, metadata.
artifacts/platform.db	SQLite user profiles and movie catalog.
artifacts/documents	Interaction history and the embedding archive.

WARNING: Rebuilding writes generated files. Back up the current artifacts and stop the API first. Do not rebuild in a directory that contains data you need to preserve without a backup.

9.1 Publish another feature version

Run python -m data.pipeline --version v2, then set RECO_FEATURE_STORE_VERSION=v2 before seeding and restarting. The --version option publishes a directory; it does not change the environment for later commands. Model weights and feature versions must remain compatible.

10. RECOMMENDATION METHODS

IDENTIFIER	BEHAVIOR AND REQUIREMENTS
als	Scores prepared latent factors with NumPy. Offline factorization uses implicit when installed; otherwise the pipeline uses SVD.
content	Compares a user content profile with item embeddings. The default features use TF-IDF and SVD.
ncf	Loads a PyTorch neural model from the configured weights directory. Requires the deep extra and compatible weights.
popularity	Uses prepared overall or category popularity. It is also the default fallback.

10.1 Train the neural model

```
python -m pip install -e ".[deep]"
python -m ai.strategies.deep_learning_strategy --train
python -m ai.strategies.deep_learning_strategy --train --epochs 3
```

Prepare the data first. Training writes ncf.pt and a report under artifacts/models/<model version>/. Select ncf with RECO_ACTIVE_STRATEGY and restart. Training options also include --batch-size, --negatives, and --learning-rate.

NOTE: Interest clicks are stored in history. The current offline pipeline reads dataset files and does not automatically incorporate that click history into retraining.

11. API CONTRACTS AND EXAMPLES

ROUTE	CONTRACT
GET /health	Service snapshot: strategy, stores, backend names, and counts.
GET /recommendations/{user_id}	Positive user ID; query k defaults to 10 and accepts 1–100.
POST /interactions	Accepts an interaction and returns HTTP 202 with event_id, accepted, and queued.
GET / and /ui/	The root redirects to the static web interface.
GET /docs and /redoc	Generated API documentation.

11.1 Interaction payload

```
{
  "user_id": 1,
  "event_type": "click",
  "item_id": 293
}
```

user_id must be positive. event_type is required and must not be blank. item_id is optional but must be positive when supplied. Other optional fields include recommended_items, model_version, metadata, and occurred_at.

A recommendation response contains user_id, k, items, strategy_id, model_version, feature_store_version, degraded, degradation_reason, and latency_ms. Each item contains item_id, rank, score, title, and categories. Scores have method-specific meanings; do not compare them as a single rating scale.

12. ERROR HANDLING AND EVENTS

CONDITION	EXPECTED BEHAVIOR
Invalid input	HTTP 400 with validation_error. FastAPI's usual 422 is overridden.
Unknown profile	HTTP 404 with user_not_found when the profile lookup confirms absence.
Cold start or feature/model problem	Usually HTTP 200 with a popularity response and degraded=true.
No usable model or fallback	HTTP 503 with model_unavailable may be returned; a failing last-resort prediction can yield an empty list.
Queue publish failure	An interaction may still be accepted with queued=false and buffered locally.

12.1 Inspect acceptance separately from storage

HTTP 202 means the request was accepted. It does not prove that the background write has finished. Check queued and the history backend, and inspect worker logs when debugging missing events. The browser's Registrado label only checks whether the HTTP response succeeded.

Linux terminal output

```
$ curl -sS "http://127.0.0.1:8765/recommendations/0?k=3"
{"code":"validation_error",
 "message":"the request does not satisfy the declared schema",
 "detail":"path.user_id: Input should be greater than or equal to 1"}
HTTP 400
```

Figure 3. Actual validation-error response, line-wrapped from the console log.

13. CONFIGURATION AND SERVICES

Settings are read from process environment variables in src/config/settings.py. No automatic .env-file loader is present. Set values before starting the service and restart after changes.

VARIABLE	DEFAULT OR EFFECT
RECO_ACTIVE_STRATEGY	als; choices include content, ncf, popularity.
RECO_FEATURE_STORE_VERSION	v1; chooses the prepared feature directory.
RECO_MODEL_VERSION	v1; used for model construction and NCF weights.
RECO_SQL_URL	Local SQLite database by default.
RECO_REDIS_URL	Empty: use an in-process cache.
RECO_RABBITMQ_URL	Empty: use an in-process queue.
RECO_MONGO_URL	Empty: store history in a local JSONL file.
RECO_LOG_LEVEL	INFO; change to DEBUG for more logging.

Other settings control dataset roots, artifact roots, model dimensions, cache lifetime, and the decorator chain. Inspect the settings module before adding overrides.

NOTE: RECO_MAX_K changes controller validation, but the HTTP schema has a fixed maximum of 100. Changing the environment alone does not extend that public limit.

14. EXTERNAL SERVICES AND CACHE BEHAVIOR

Redis, RabbitMQ, MongoDB, and PostgreSQL adapters are present, but the project notes do not report real-server validation. The local fallbacks are the normal demonstration path. Test each external service before relying on it (Michou001, 2026).

14.1 RabbitMQ worker

```
python -m messaging.feedback_history_service
python -m messaging.feedback_history_service --once
```

Run the worker with the same RECO_RABBITMQ_URL and queue name as the API. The API starts its own worker thread only when the selected queue is in-memory. A separate process cannot consume another process's memory queue.

14.2 Verify the selected backend

Check /health after startup. Redis and RabbitMQ connection failures can select local fallbacks. A configured URL alone is not proof that an external backend is active. PostgreSQL also needs the matching SQLAlchemy database driver; it is not supplied by the listed extras.

14.3 Invalidate stale models and rankings

ModelFactory reuses instances by strategy ID and model version. The ranking cache key contains user ID, k, and model version. It does not include the strategy ID. Avoid sharing one Redis namespace across different strategies with the same version until this key design is addressed.

Restart processes after publishing new data or changing configuration. Give model releases distinct versions and coordinate cache invalidation. Do not overwrite active model files while the service is using them.

15. ADD OR CHANGE A FEATURE

15.1 Add a recommendation strategy

Create a module under src/ai/strategies. Implement IRecommendationStrategy and register a unique identifier. Import the new module from src/ai/strategies/__init__.py so registration runs. No new factory branch is needed.

```
from ai.interfaces import IRecommendationStrategy, ScoredItem
from ai.model_factory import register_strategy

@register_strategy("example")
class ExampleStrategy(IRecommendationStrategy):
    def predict(self, user_features, k=10):
        return [ScoredItem(item_id=293, score=1.0)][:k]
```

NOTE: This minimal example demonstrates the contract only. A real strategy must load compatible artifacts, rank valid items, exclude seen items when required, and handle missing features.

Override build(settings) to load artifacts once. Return ordered ScoredItem objects and raise the project's domain errors when the strategy cannot run. Add type annotations and tests before using the example in the project.

15.2 Change an endpoint or storage adapter

Define request and response changes in presentation/schemas.py, then update the route and application service. Keep scoring out of routes. Implement persistence interfaces for new storage backends, wire them in the composition root, and extend the relevant tests.

16. TESTS AND MEASURED PERFORMANCE

python -m pytest -ra

python -m ruff check .

python -m black --check .

python -m mypy

python benchmarks/latency_p95.py --requests 100 --json

Tests create small temporary stores. Run targeted tests during development and the broader suite before handoff (pytest developers, n.d.). In this Linux run, 256 tests passed and one optional PyTorch test module was skipped. The run produced 41 warnings; it was not a warning-free or complete deep-learning test.

Linux terminal output

```
$ python -m pytest -ra
256 passed, 1 skipped, 41 warnings in 11.84s
$ python -m ruff check .
All checks passed!
$ python -m black --check .
71 files would be left unchanged.
$ python -m mypy
Success: no issues found in 39 source files
```

Figure 4. Selected lines from actual Linux validation output.

The 100-request controller benchmark returned p95 = 1.016 ms with als, caching enabled, and no errors or degraded responses. This is a local, sequential controller measurement, not an HTTP load test or a promise for other machines.

17. DEBUGGING AND DELIVERY

17.1 Common development problems

SYMPTOM	CHECK
module not found	Activate the intended .venv and rerun the editable install from the repository root.
missing data or weights	Check feature and model versions, then rebuild or restore compatible artifacts.
unexpected popularity results	Read degraded and degradation_reason, then inspect startup and inference logs.
tests skipped	Install the relevant optional extra. A skipped module is not evidence that its features passed.
different results after reinstall	Dependencies are not locked. Record python and package versions from the working environment.

17.2 Before releasing a change

Review the diff, run relevant tests and checks, confirm both success and failure API paths, and record the source revision, dataset version, model version, and dependency versions. Keep a recoverable copy of the previous artifacts before migration.

WARNING: Authentication, authorization, rate limiting, container orchestration, and a release pipeline are not implemented in this revision. Treat public deployment as additional engineering work. Remove --reload and validate external services before any deployment beyond local development.

The validation in this manual used the dev and als extras. PyTorch, external service connections, and Windows/macOS execution were not validated during this run.

18. REFERENCES

FastAPI. (n.d.). *Run a server manually.* https://fastapi.tiangolo.com/deployment/manually/

Git. (n.d.). *git-clone documentation.* https://git-scm.com/docs/git-clone

GroupLens Research. (n.d.). *MovieLens datasets.* https://grouplens.org/datasets/movielens/

Michou001. (2026). *Intelligent-Recommendation-Plataform* (Version 0.2.0, revision 00cabc2) [Source code]. GitHub. https://github.com/Michou001/Intelligent-Recommendation-Plataform

pytest developers. (n.d.). *How to invoke pytest.* https://docs.pytest.org/en/stable/how-to/usage.html

Python Software Foundation. (n.d.). *venv — Creation of virtual environments.* Python documentation. https://docs.python.org/3/library/venv.html